

# AP4

Mariam Tchubabria, Kutaisi International University, Fall 2025

## Overview

In this project, I worked on visualizing how satellites cover different parts of the Earth using **Voronoi diagrams** and **Delaunay triangulation**. I took data from [List Of Argentina Satellites](#), We have 35 satellite. Each satellite acts like a central point that "owns" the closest region around it. I compared how these regions change depending on the type of distance used (the  $L_p$  norm). I implemented both the exact Euclidean version (using SciPy) and a grid-based approach that works for any norm.

## Theory

**Voronoi Diagram:** A Voronoi diagram splits space into areas based on which satellite (point) is closest. If we have satellites  $S = \{s_1, \dots, s_n\}$ , then the region for  $s_i$  is all the points that are closer to  $s_i$  than to any other:

$$V(s_i) = \{x \in \mathbb{R}^2 : \|x - s_i\|_p \leq \|x - s_j\|_p, \forall j \neq i\}, \quad \|x - s_i\|_p = \left( \sum_{k=1}^d |x_k - s_{i,k}|^p \right)^{1/p}$$

In simple terms, every satellite gets its own "coverage zone." The boundaries show where two satellites have equal influence.

**Delaunay Triangulation:** This connects satellites with lines so that no satellite lies inside any other satellite's triangle circle. It's kind of the "inverse" of the Voronoi diagram. Two satellites are connected if their Voronoi regions touch. It helps visualize which satellites are direct neighbors.

**Distance Norms:** The type of distance changes the shape of regions:

- $L_1$  (Manhattan): diamond-shaped zones
- $L_2$  (Euclidean): circular zones (the usual one)
- $L_\infty$  (Chebyshev): square-shaped zones

## Implementation

I used two main methods:

- **Method 1 – Exact Euclidean:** Used `scipy.spatial.Voronoi` and `Delaunay` for the perfect  $L_2$ -based diagram.
- **Method 2 – Grid-Based  $L_p$ :** For other norms, I generated a 1000×1000 grid of points and computed distances to each satellite using the chosen  $p$  value. Then each grid point was assigned to the closest satellite.

The input data came from a CSV file with satellite names, coordinates, and signal strengths. The program automatically adjusts the plot area with a small margin so everything fits nicely.

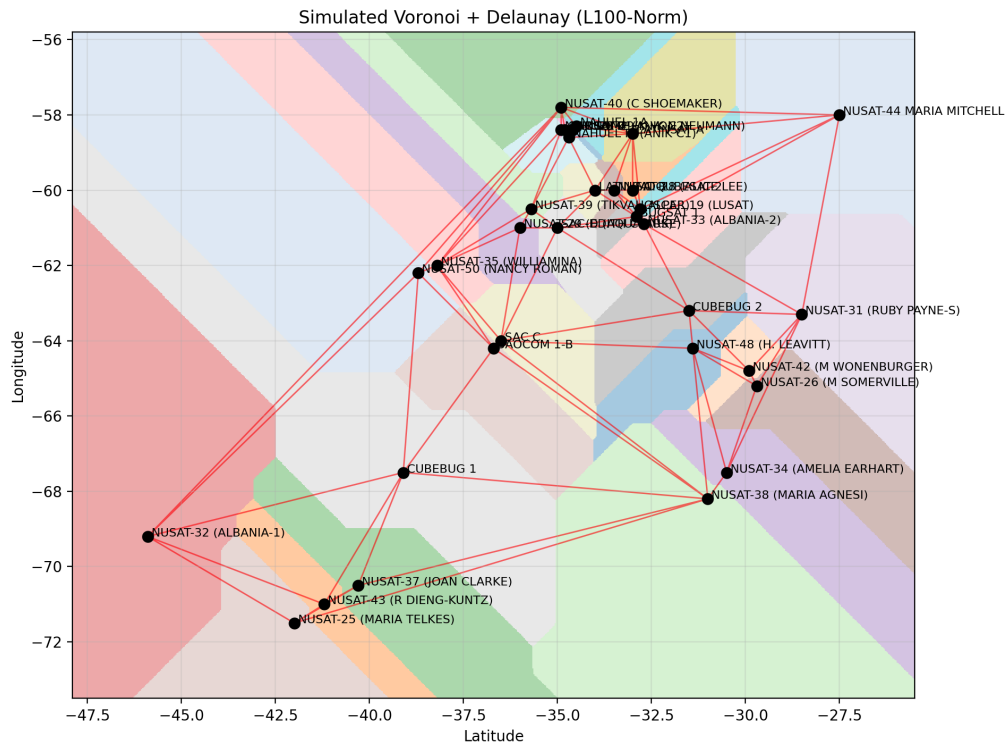
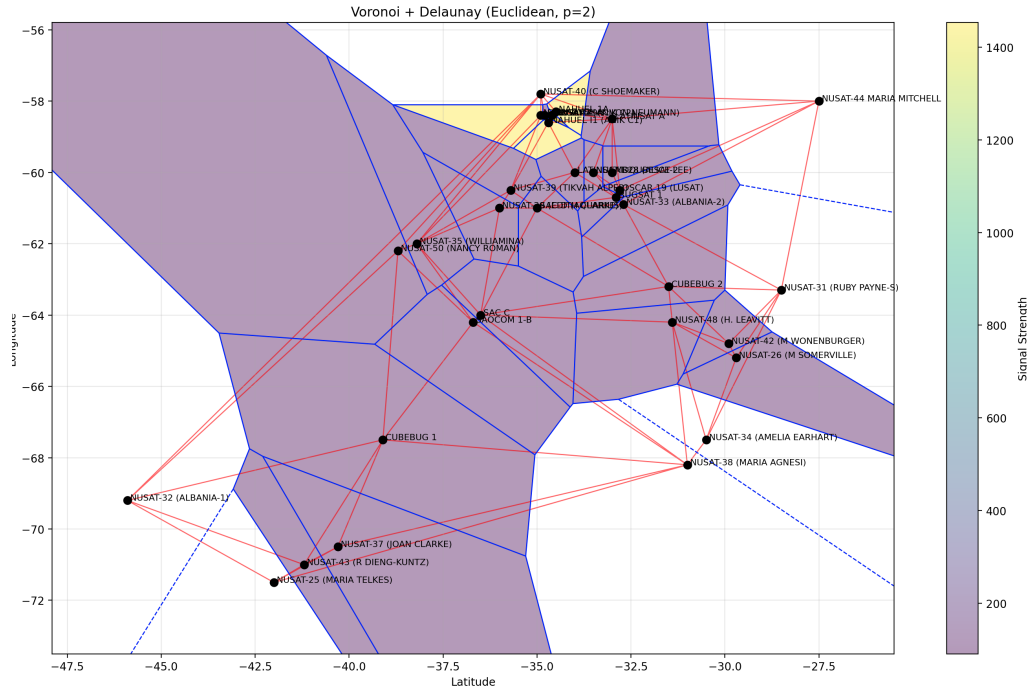
## Observations

- When  $p = 2$ , both methods (exact and grid-based) give almost identical results.
- The grid resolution of 1000×1000 was smooth enough to look realistic.
- Changing  $p$  changes the shape of the regions. Higher norms tend to give satellites more square liked regions.
- Delaunay triangulation clearly shows which satellites are "neighbors" that share borders.
- In builtin diagram the color of each region matches the satellite's signal strength, so stronger satellites appear brighter.

## Conclusions

- I successfully implemented both exact and simulated Voronoi diagrams.
- The grid-based method works well for any  $p$ -norm, even though it's slower.
- The Delaunay triangulation adds another layer of information by showing direct connections between satellites.

## Results

Figure 1: Grid-based  $L_{100}$ -norm Voronoi and Delaunay TriangulationFigure 2: Grid-based  $L_2$ -norm Voronoi and Delaunay triangulation